



数字图像处理

Digital Image Processing

第07章 二值图像形态学处理 (教材的第9章)

张朝晖

河北师范大学

2024年9月~2024年12月



河北师范大学软件学院
Software College of Hebei Normal University

主要内容

7.1 预备知识

7.2 腐蚀和膨胀

7.3 开启与闭合

7.4 击中击不中变换

7.5 二值形态学基本应用



河北师范大学软件学院
Software College of Hebei Normal University

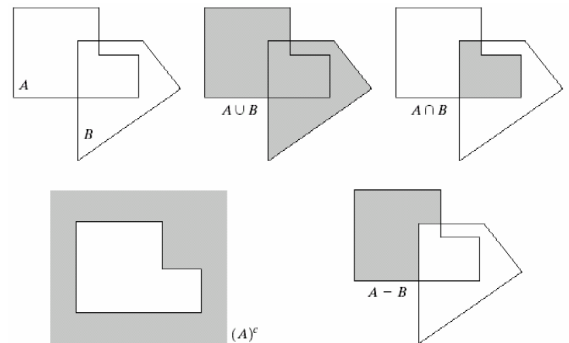
2

1. 形态学概况

- 形态学是生物学一个分支，研究动物和植物结构。
数学形态学（也称图像代数），是以形态为基础对图像进行分析的数学工具
- 数字图像形态学处理目的--研究数字图像中物体目标的结构及拓扑关系
- 基本思想：用具有一定形态的**结构元素**去度量和提取图像的**对应形状**，以达到对图像分析和识别的目的
- 形态学图像处理的教学基础是**集合论**，基本运算：**膨胀、腐蚀、开操作和闭操作**

2. 集合的基本运算

(1) 集合的并、交、补、差



河北师范大学软件学院
Software College of Hebei Normal University

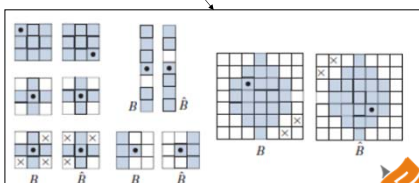
(2) 集合的反射、平移

集合B的平移

$$(B)_{z=(z_1, z_2)} = \{c | c = b + z, b = (x, y) \in B\}$$

集合B的反射

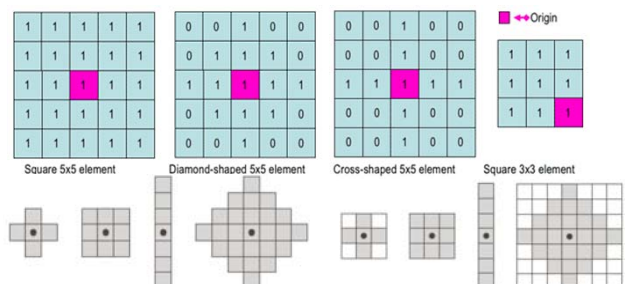
$$\hat{B} = \{w | w = -b, b = (x, y) \in B\}$$



河北师范大学软件学院
Software College of Hebei Normal University

3. 形态学图像处理的“探测器”——结构元(结构算子)

---结构元的大小、形状、原点位置



河北师范大学软件学院
Software College of Hebei Normal University

图像形态学处理表现为一种邻域运算形式:

利用“结构元”(Structure Element)
 的特殊邻域(一定大小、一定形状、不同的
 原点),与输入图像(二值图像、灰度图像)
 中各个像素邻域进行特定的逻辑运算,逻辑
 运算的结果为输出图像的相应像素取值。

形态学运算的效果取决于结构元的大小、
 内容以及逻辑运算的性质。

主要内容

7.1 预备知识

7.2 腐蚀(erosion)和膨胀(dilation)

7.3 开启与闭合

7.4 击中击不中变换

7.5 二值形态学基本应用

1. 腐蚀(erosion)

定义1--对二值图像中的目标A进行腐蚀(常采用该方式)

$$A \ominus B = \{z | (B)_z \subseteq A\} = \{z | (B)_z \cap A^c = \emptyset\}$$

$(B)_z$ ----结构元B以目标A中的z为原点

定义2--对二值图像I中的所有目标进行腐蚀

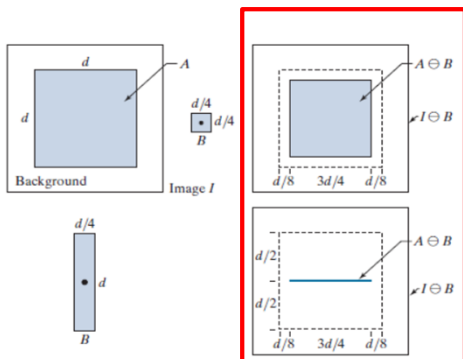
$$I \ominus B = \{z | (B)_z \subseteq A \text{ 并且 } A \subseteq I\} \cup \{A^c | A^c \subseteq I\}$$

$$A \ominus B$$

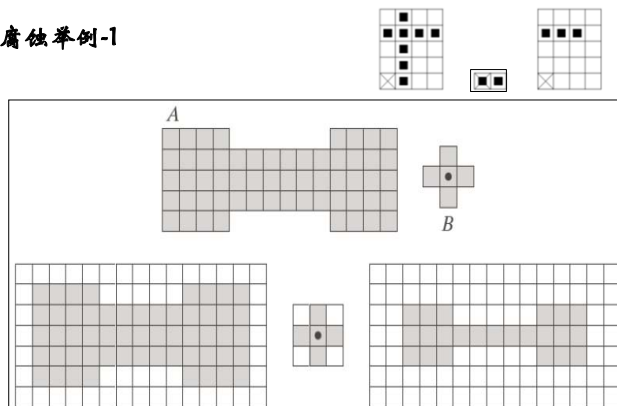
每当在目标A中找到一个与结构元素B相同的子图像
 时,就把该子图像中与B的原点位置对应的那个像素
 位置标注为输出图像的目标像素。

目标A上标注出的所有这样的像素组成的集合,即为
 目标A腐蚀后的输出结果。

腐蚀示例

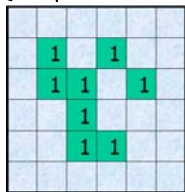


腐蚀举例-1

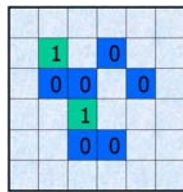


腐蚀举例-2

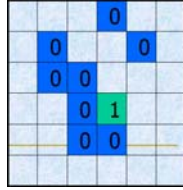
结构元B



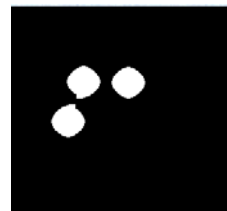
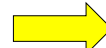
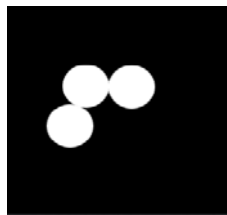
输入图像A



输出图像 $A \ominus B$



腐蚀举例-3

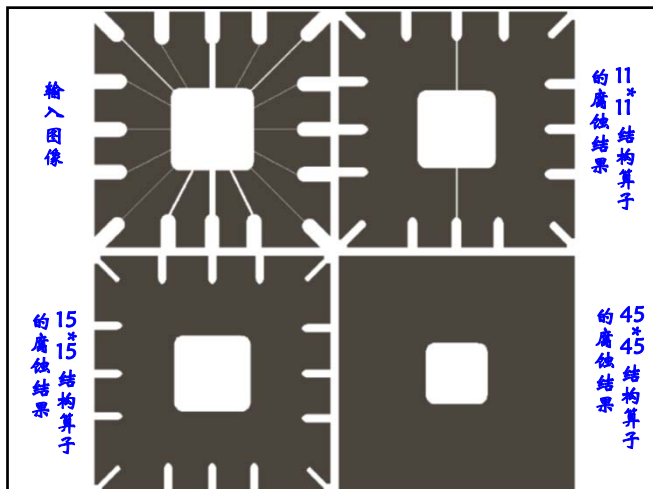


腐蚀运算可以消除图像中目标之间的粘连



河北师范大学软件学院
Software College of Hebei Normal University

14



腐蚀作用

- 收缩或细化二值图像中的目标区域，并能断开连接
- 滤波--滤除那些小于结构算子的图像目标
- 腐蚀运算的效果与结构元素的形状(矩形、圆形、菱形等)、大小、原点位置的选取有关。



河北师范大学软件学院
Software College of Hebei Normal University

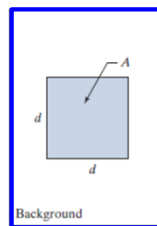
2. 膨胀(dilation)

$$A \oplus B = \{z \mid (\hat{B})_z \cap A \neq \emptyset\} = \{z \mid [(\hat{B})_z \cap A] \subseteq A\}$$

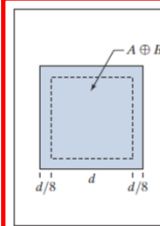
$(\hat{B})_z$ ----结构元B关于原点的反射 $\hat{B}$ 平移至目标A中z处

- 每当结构元素 在目标A上平移后，结构元素与其覆盖的子图像中至少有一个目标元素相交，就将A中与结构元素的原点对应的那个位置的像素值置为“目标”，否则置为“背景”。
- A上标注出的所有这样的“目标”像素组成的集合，即为对A膨胀运算的结果。

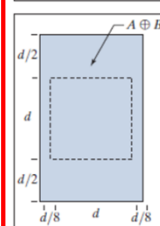
膨胀示例



$$\hat{B} = B$$



$I \oplus B$



$I \oplus B$



河北师范大学软件学院
Software College of Hebei Normal University

17



河北师范大学软件学院
Software College of Hebei Normal University

EROSION 腐蚀
 DILATION 膨胀

河北师范大学软件学院
 Software College of Hebei Normal University

膨胀举例

结构元B及其 $\hat{B}$

输入图像A

输出图像 $A \oplus B$

河北师范大学软件学院
 Software College of Hebei Normal University

膨胀运算可以：连接邻近目标
 填充目标区域小孔

膨胀运算，可以桥接
 二值字符图像的单个
 字符裂缝

河北师范大学软件学院
 Software College of Hebei Normal University

Historically, certain computer programs were written using only two digits rather than four to define the applicable year. Accordingly, the company's software may recognize a date using "00" as 1900 rather than the year 2000.

Historically, certain computer programs were written using only two digits rather than four to define the applicable year. Accordingly, the company's software may recognize a date using "00" as 1900 rather than the year 2000.

膨胀运算，可以桥接
 二值字符图像的单个
 字符裂缝

河北师范大学软件学院
 Software College of Hebei Normal University

膨胀应用：借助膨胀运算，获取封闭轮廓

左：某灰度图像的边缘检测结果(二值边缘图像)
 中：二值图像的膨胀，得到闭合的“厚”轮廓
 右：轮廓内填充，得到目标区域

河北师范大学软件学院
 Software College of Hebei Normal University

膨胀作用

- “扩张”或“粗化”二值图像中的目标区域
- 填充目标区域中的小孔、桥接缝隙
- 膨胀运算的效果与结构元素的形状(矩形、圆形、菱形等)、大小、原点位置的选取有关。

河北师范大学软件学院
 Software College of Hebei Normal University

3. 腐蚀与膨胀的对偶性

$$[A \ominus B]^c = A^c \oplus B^c$$

$$[A \oplus B]^c = A^c \ominus B^c$$

➤ 对图像中目标区域的膨胀运算，相当于对图像中背景区域的腐蚀运算

➤ 对图像中目标区域的腐蚀运算，相当于对图像中背景区域的膨胀运算

主要内容

7.1 预备知识

7.2 腐蚀(erosion)和膨胀(dilation)

7.3 开启(opening)与闭合(closing)

7.4 击中击不中变换

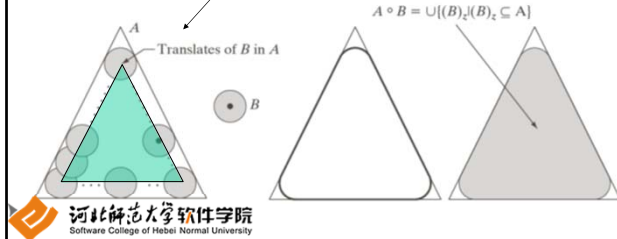
7.5 二值形态学基本应用

1. 开启(opening)

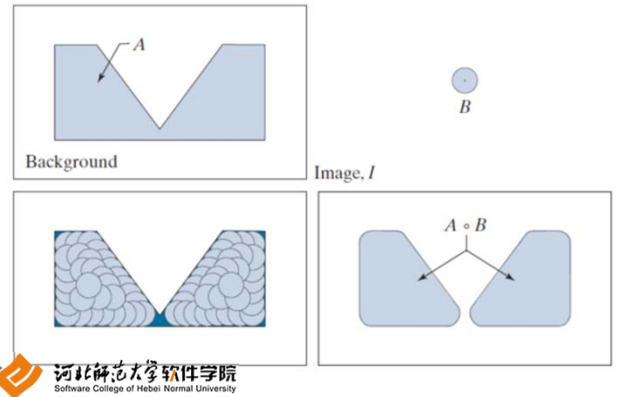
$$A \circ B = (A \ominus B) \oplus B$$

$$A \circ B = \bigcup \{ (B)_z \mid (B)_z \subseteq A \}$$

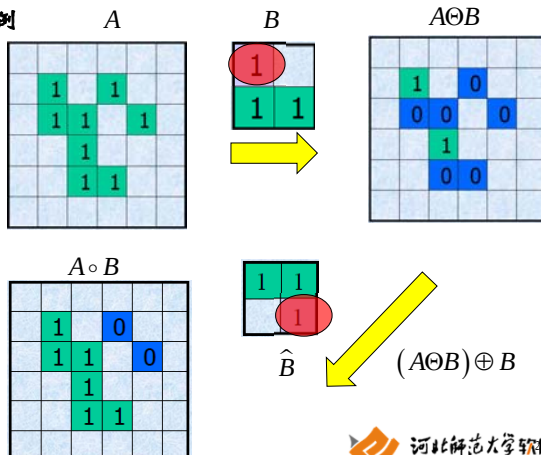
➤ 以结构元素B对集合A的开启运算，就是：先对A进行腐蚀，再对腐蚀结果进行膨胀



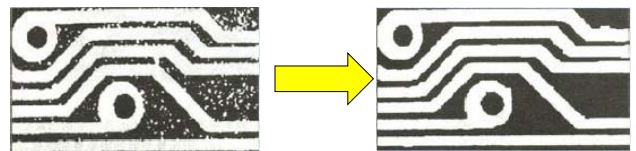
$$A \circ B = \bigcup \{ (B)_z \mid (B)_z \subseteq A \}$$



举例



开运算举例



借助开运算，对二值印刷电路板图像进行滤波

开运算作用

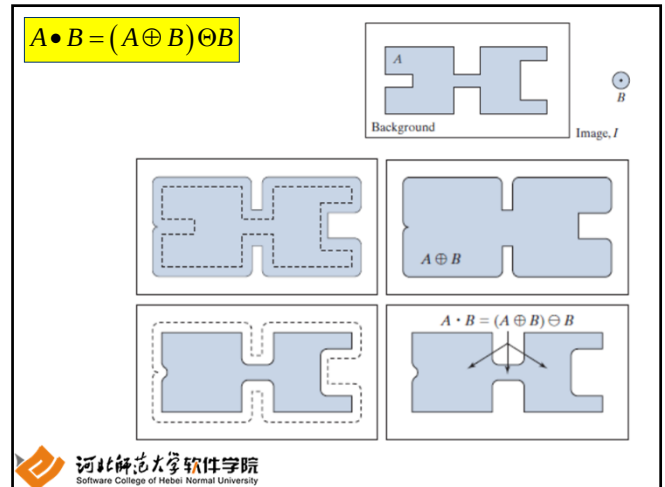
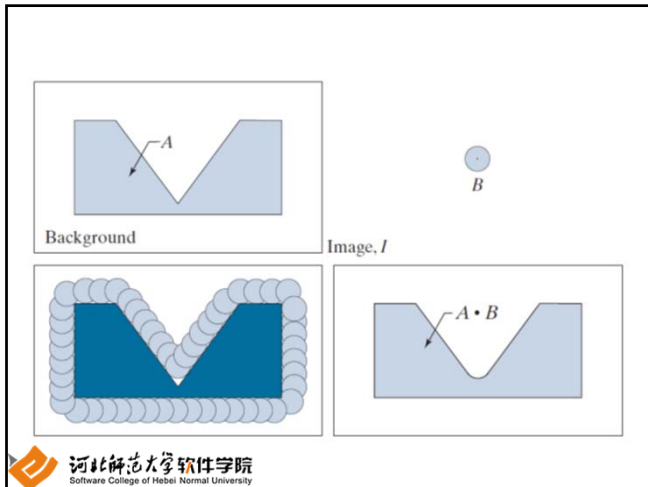
- 平滑物体的轮廓、磨光目标对象外凸的毛刺
- 断开较窄的狭颈并消除细的突出物
- 去除小的目标区域(滤波)

2. 闭合(closing)

$$A \bullet B = (A \oplus B) \ominus B$$

$$A \bullet B = \left[\bigcup \{ (B)_x \mid (B)_x \cap A = \Phi \} \right]^C$$

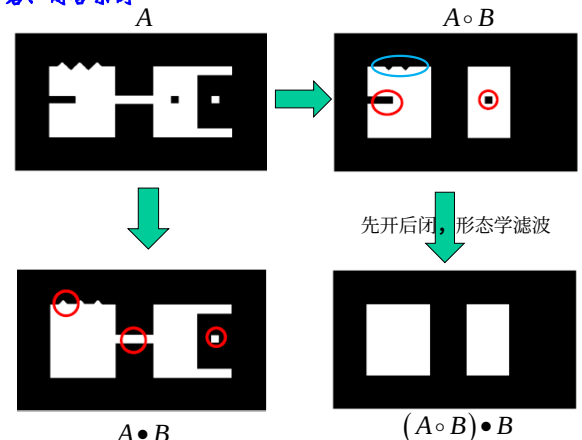
- 以结构元素B对集合A的闭合运算，就是：先对A进行膨胀，再对膨胀结果进行腐蚀

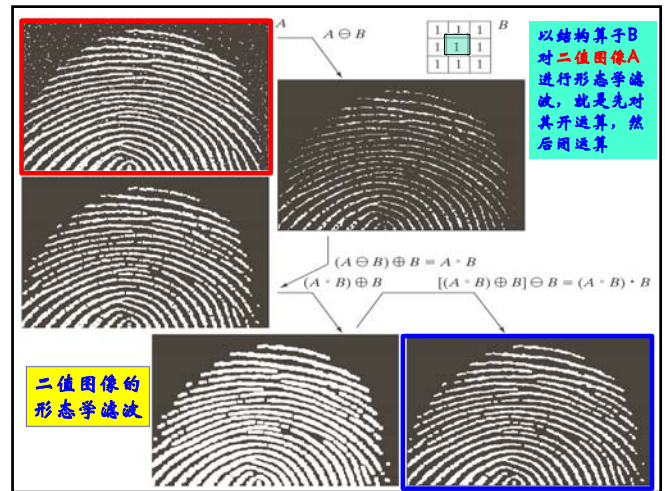
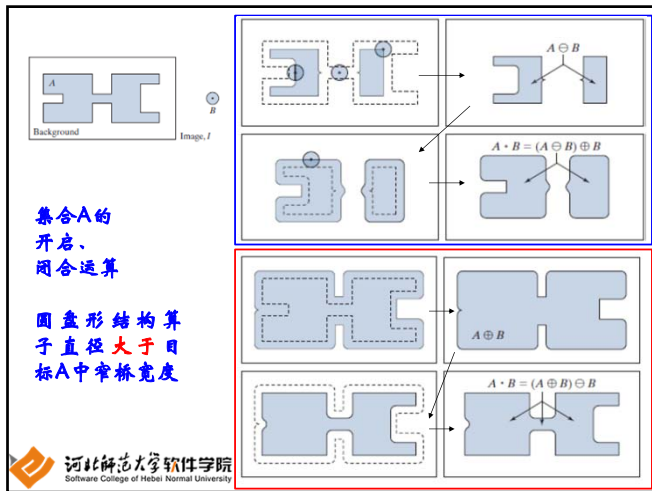


闭合运算的作用

- 填充目标区域内细小空洞
- 连接邻近物体
- 平滑其边界的同时并不明显改变其面积
- 磨平目标区域内凹的毛刺

开启、闭合示例





主要内容

- 7.1 预备知识
- 7.2 腐蚀(erosion)和膨胀(dilation)
- 7.3 开启(opening)与闭合(closing)
- 7.4 击中击不中变换
- 7.5 二值形态学基本应用

河北师范大学软件学院
Software College of Hebei Normal University

39

➤ “击中-击不中变换(Hit or Miss Transform, HMT)”是用于二值图像中目标形状检测基本工具。

➤ 若给定的二值图像中含指定“形状与大小”目标对象(由前景像素与背景像素的几何排列决定)，则返回相应形状对象的位置

----实质：形状匹配

河北师范大学软件学院
Software College of Hebei Normal University

40

➤ 令二值图像I由前景(A)及背景组成。

➤ HMT同时使用两种结构算子 B_1 B_2 进行形状检测：
 B_1 ----用于前景中形状检测
 B_2 ----用于背景中形状检测

➤ 关于二值图像I的HMT运算：

$$I \circledast B_{1,2} = \{z | (B_1)_z \subseteq A \text{ 并且 } (B_2)_z \subseteq A^c\}$$

$$= (A \ominus B_1) \cap (A^c \ominus B_2)$$

其它类似的定义：

$$I \circledast B = (A \ominus D) \cap (A^c \ominus (W - D))$$

其中 $B = (D, W - D)$

河北师范大学软件学院
Software College of Hebei Normal University

41

例：采用“击中-击不中变换”在集合A中检测D形状对象

(1)根据感兴趣的目標对象D($d \times d$ 大小)构造结构算子 B_1 与 B_2 (即： $W - D$)

$(d+2) \times (d+2) = (2a+3) \times (2a+3)$

该算子的中间区域(即 $d \times d$ 部分)为有效部分，用于探测图像中的前景像素；因对其余四边取值不做要求，因此探测时只关注算子覆盖区域的 $d \times d$ 部分

$(d+2) \times (d+2) = (2a+3) \times (2a+3)$

该算子的厚度为1个像素的四边区域为有效部分，用于探测原始图像中的背景像素(即：补集图像的前景像素)；因对其中心区域的 $d \times d$ 部分取值不做要求，因此探测时，只关注该模板覆盖区域的四个边

河北师范大学软件学院
Software College of Hebei Normal University

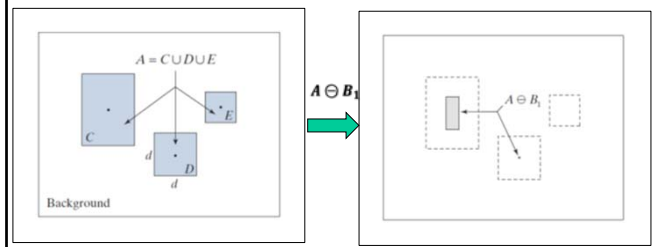
(2)对原二值输入图像I前景区域 ($A = C \cup D \cup E$) 执行腐蚀

具体实现细节:

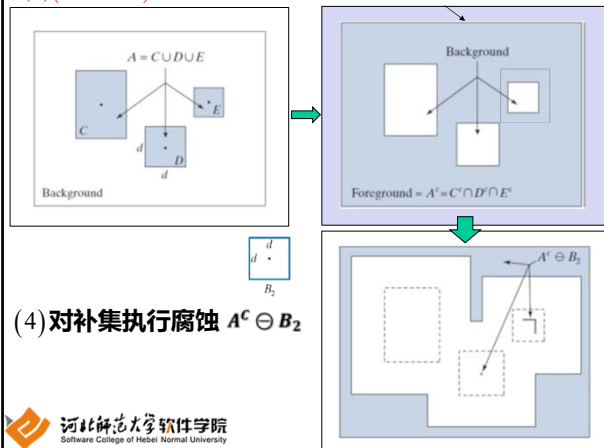
首先,对原始图像四周增加背景边,加边厚度与两个模板 B_1, B_2 的大小及模板原点中心位置有关。

然后,在加边基础上,执行腐蚀 $A \ominus B_1$

例:本任务中,对原始图像上下左右加边厚度为 $a+1$

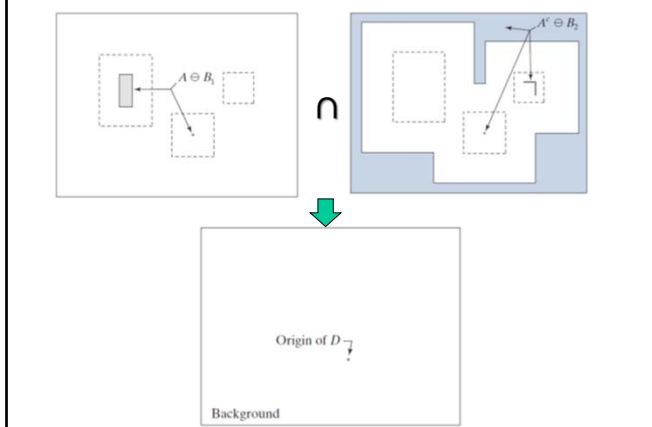


(3)(加边后的)二值输入图像取反,得 $I^C \Rightarrow$ 集合A的补集 A^C



(4)对补集执行腐蚀 $A^C \ominus B_2$

(5)两种腐蚀结果去边,然后 $I \ominus B_{1,2} = (A \ominus B_1) \cap (A^C \ominus B_2)$



例:采用“击中-击不中变换”在图像A中检测B1形状对象

(1)根据感兴趣的目标,构造结构算子 B_1 与 B_2

感兴趣的形状对象 B_1



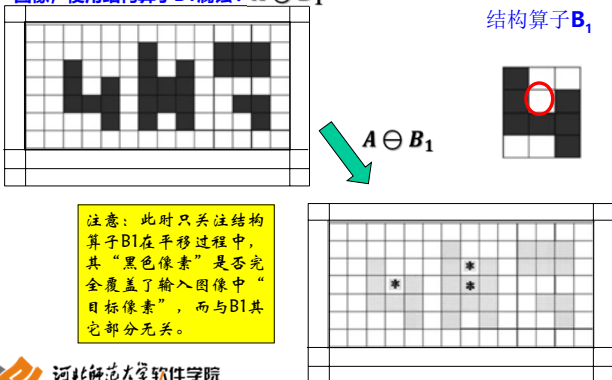
使用 B_1 ,只关注结构算子 B_1 在原始输入图像中平移过程中,其“黑色像素”是否完全覆盖了输入图像中“目标像素”,而不关注 B_1 其它部分覆盖的像素。

B_1 各元素取反,得结构算子 B_2



使用 B_2 ,只关注结构算子 B_2 在输入图像的补集图像中平移过程中,其“黑色像素”是否完全覆盖了与输入图像的“背景像素”对应的部分,而不关注 B_2 其它部分覆盖的像素。

(2)对二值输入图像A首先加背景边,该图像的顶、底、左、右的加边厚度分别为1像素、2像素、1像素、1像素;然后对加边后的图像,使用结构算子 B_1 腐蚀: $A \ominus B_1$

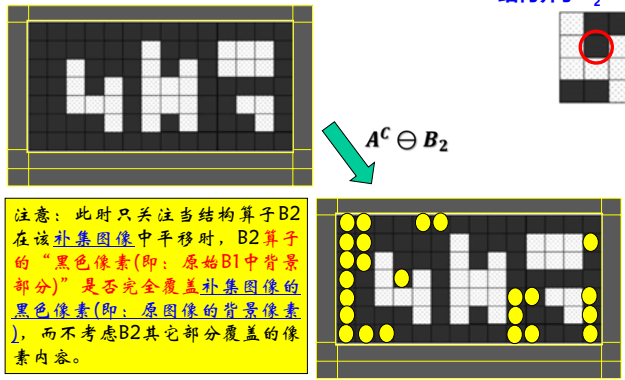


注意:此时只关注结构算子 B_1 在平移过程中,其“黑色像素”是否完全覆盖了输入图像中“目标像素”,而与 B_1 其它部分无关。

(3)对加背景边后的二值输入图像A取反得 A^C

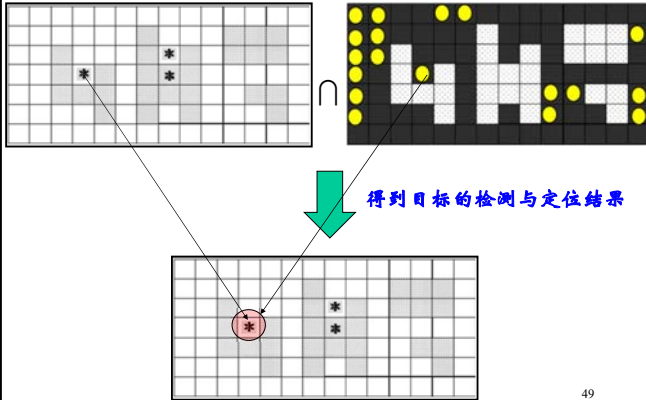
然后使用结构算子 B_2 执行腐蚀运算: $A^C \ominus B_2$

结构算子 B_2



注意:此时只关注当结构算子 B_2 在该补集图像中平移时, B_2 算子的“黑色像素(即:原始 B_1 中背景部分)”是否完全覆盖补集图像的背景像素(即:原图像的背景像素),而不考虑 B_2 其它部分覆盖的像素内容。

(4)对两种腐蚀结果去边后，执行运算： $(A \ominus B_1) \cap (A^c \ominus B_2)$



49

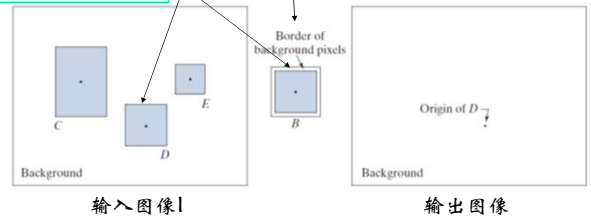
关于击中-击不中变换的更简单的实现：

针对感兴趣的形状对象D，对其加边(在上下左右四个方向各增加宽度为1个像素的背景边)，得到特殊的结构算子B，将HMT定义如下：

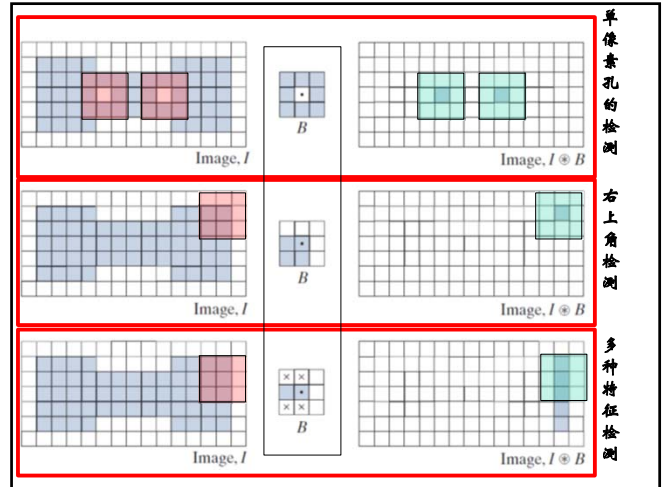
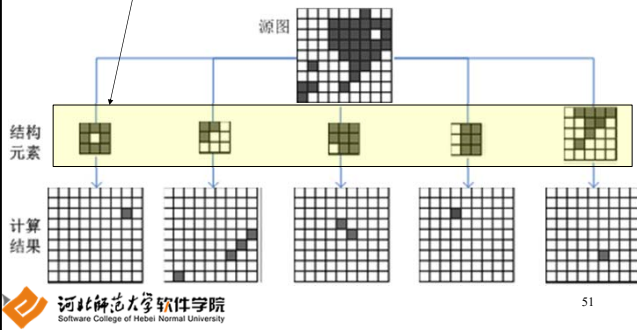
结构算子是否加边，与该算子元素是否完全为目标像素有关。若该算子元素均对应图像目标背景，则需要加边。

$$I \oplus B = \{x | (B)_x \subseteq I\}$$

注意：原始图像处理前也应加边，该图像在不同方向的加边厚度取决于(加边后)算子中心到其各边的距离处理后，再去边。



本页及下一页的“HMT”运算，使用了“未加边”的结构算子



例：采用击中-击不中变换检测图像中字符“e”

二值输入图像

结构算子

The term watershed
refers to a ridge that ...

e

检测结果

The term watershed
refers to a ridge that ...

主要内容

- 7.1 预备知识
- 7.2 腐蚀(erosion)和膨胀(dilation)
- 7.3 开启(opening)与闭合(closing)
- 7.4 击中击不中变换
- 7.5 二值形态学基本应用

目标区域的边界提取

孔洞填充

连通分量提取

1. 边界提取 (Boundary Extraction)

External Boundary

$$\beta(A) = A \oplus B - A$$

Internal Boundary

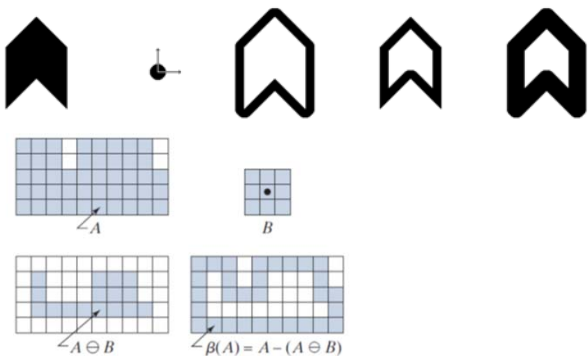
$$\beta(A) = A - A \ominus B$$

Morphological Gradient

$$\beta(A) = A \oplus B - A \ominus B$$

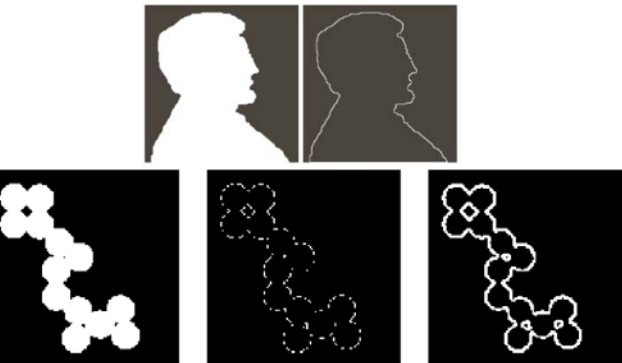
其中 $\left\{ \begin{array}{l} A \text{--感兴趣的集合对象} \\ B \text{--结构算子} \\ \beta(A) \text{--集合A的边界} \end{array} \right.$

例：利用结构算子提取黑色目标区域不同类型的边界



例：采用3*3方形结构算子提取白色目标区域边界

注意：区域边界为单像素宽



2. 孔洞填充 (Hole filling)

孔洞--由前景像素相连接的边界包围的背景区域

$$X_k = (X_{k-1} \oplus B) \cap A^C \quad k = 1, 2, 3, \dots$$

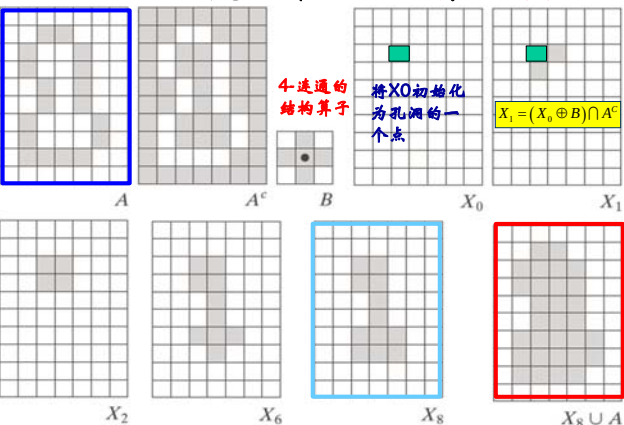
迭代填充，直到 X_k 与 X_{k-1} 一致

本质：有条件地将背景像素置为前景像素。

经孔洞填充后的前景对象： $A_{out} = X_k \cup A$

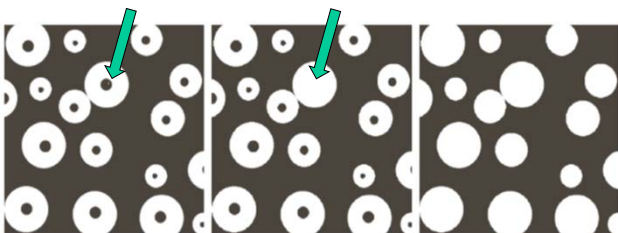
其中 $\left\{ \begin{array}{l} A \text{--含空洞的集合对象} \\ B \text{--结构算子} \\ X_0 \text{--嵌在集合A的初始孔洞 (设置初始点)} \\ X_k \text{--经k步迭代填充后的孔洞集合} \end{array} \right.$

例：目标的孔洞填充示例(8-连通的边界)—条件膨胀



孔洞填充示例

二值输入图像 某指定区域的填充结果 所有区域的孔洞填充



3. 连通分量的提取 (Extracting connected components)

$\begin{cases} \text{do } X_k = (X_{k-1} \oplus B) \cap A & k=1, 2, 3, \dots \\ \text{till } X_k = X_{k-1} \end{cases}$

有条件的迭代膨胀，直到 X_k 与 X_{k-1} 一致

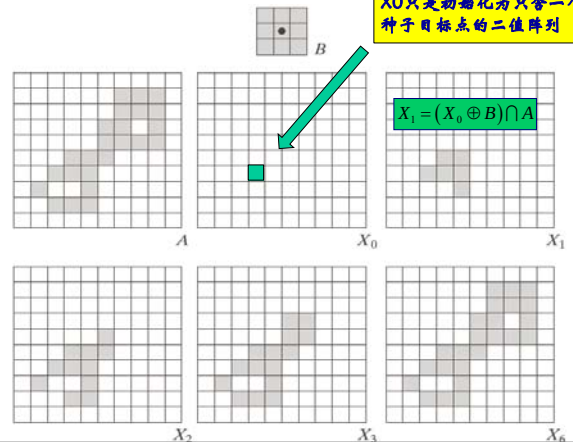
最终输出连通分量： X_k

其中

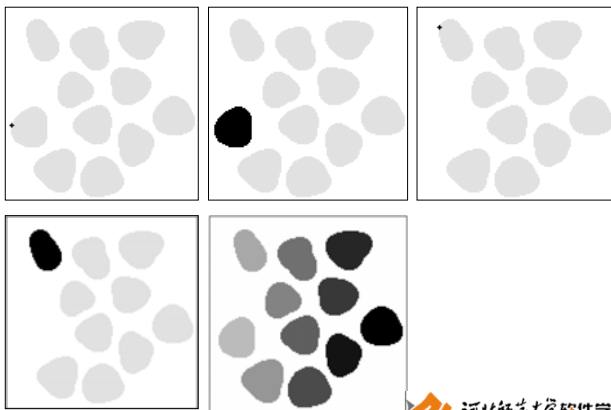
- A --包含一个或多个连通分量的集合对象
- B --结构算子
- X_0 --连通分量初始阵列(一个或多个种子点)
- (只有连通分量初始种子点处置为前景)
- X_k --经 k 步迭代后的连通分量

例：单个连通分量提取

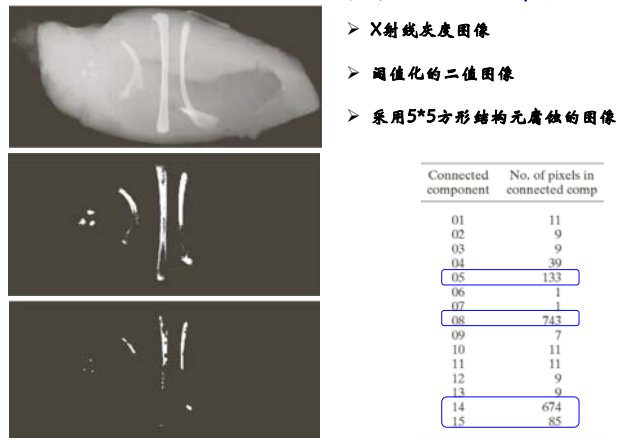
图只有一个连通分量，所以 X_0 只是初始化为只有一个种子目标点的二值阵列



例：多个连通分量提取



例：采用连通分量检测肌肉X射线图像所含的碎骨



4. 边界清除

$I(x, y)$ --二值输入图像(背景为0)
 设 F --标记图像
 X --边界清除后的输出图像

并且 $F(x, y) = \begin{cases} I(x, y) & \text{若 } (x, y) \text{ 在 } I \text{ 的边界上} \\ 0 & \text{其它} \end{cases}$

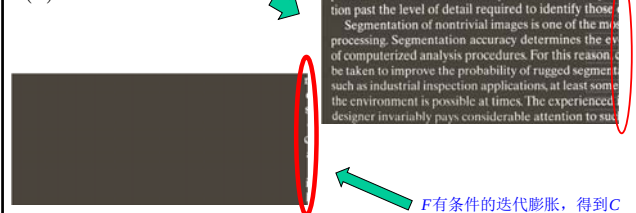
以 F 初始化 X_0 , $\begin{cases} \text{do } X_k = (X_{k-1} \oplus B) \cap I & k=1, 2, 3, \dots \\ \text{till } X_k = X_{k-1} \end{cases}$

有条件的迭代膨胀，直到 X_k 与 X_{k-1} 一致

最终得到： $C=X_k$

则 $X = I - C$

(1) 原始二值输入图像I



(2) 标记图像F

$F(x, y) = \begin{cases} I(x, y) & \text{若 } (x, y) \text{ 在 } I \text{ 边界上} \\ 0 & \text{其它} \end{cases}$

(3) 输出图像 $X = I - C$

结构元 B 为 3×3

ponents or broken connection paths. There is no position past the level of detail required to identify those Segmentation of nontrivial images is one of the most processing. Segmentation accuracy determines the evolution of computerized analysis procedures. For this reason, be taken to improve the probability of rugged segment such as industrial inspection applications, at least some the environment is possible at times. The experienced designer invariably pays considerable attention to such

本章小结

二值图像形态学：

腐蚀、膨胀、

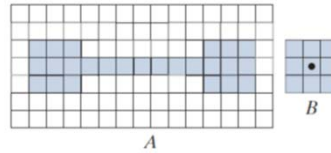
开、闭、

形态学滤波

应用：目标区域的边界提取、孔洞填充

连通分量提取

边界清除



对于左图所示的二值图像中的目标A，以及结构算子B，若对A分别执行如下操作，请标记每种处理后得到的输出结果：

(1) $A \ominus B$ (2) $A \oplus B$ (3) $A \circ B$ (4) $A \cdot B$

(5) $A \oplus B - A \ominus B$

(6) $A \oplus B - A$

(7) $A - A \ominus B$

(8) 若将结构算子B替换为3×3的十字形，指出上述各环节输出结果。

(9) 若将结构算子B替换为3×1的条形，指出上述各环节输出结果。

注意不同结构算子导致的处理结果的区别。



给定如图所示的二值图像，其中图像中的集合A由白色前景像素组成（结构算子B除外）。假设图中所示的圆点型结构算子B尺寸足够合适（即大于图像中任意白色或黑色随机小块，同时明显小于白色矩形区域）。若对上述图像处理，依次执行以下步骤，指出各步骤处理后的结果：

(1) $C = A \ominus B$; (2) $D = C \oplus B$;

(3) $E = D \oplus B$; (4) $F = E \ominus B$

